



Home Work

DevOps

HW_L3_02_Revers eProxy

۱. فایل تمرین را در پنل خود آپلود کنید.

۲. title فایل تمرین به صورت (نام تمرین+نام و نام خانوادگی) به انگلیسی باشد.

نمونه: AmirMohammadMohammadi_reverseproxy_02

۳. در صورتی که سوال و یا ابهامی دارید در گروه چت تلگرامی بپرسید.

تمرین Reverse Proxy

مجموع امتیاز: سی نمره

تمام سناریوها با استفاده از Docker و Docker Compose اجرا می شوند

برای هر سوال فایل و کانفیگ نمونه داده شده تا دقیقاً همان را بسازید

تحويل يك پوشه كامل شامل پوشه‌های هر سناریو، کانفیگ‌ها، docker-compose.yml و خروجی دستورها

ساختار پیشنهادی پوشه کار:

reverse_proxy_homework/

├── 01_docker_setup/

├── 02_nginx_basics/

├── 03_nginx_certs/

├── 04_nginx_admin/

├── 05_nginx_mtls/

├── 06_haproxy/

└── 07_traefik/

مجموعه سوالات اول: آماده‌سازی Docker و محیط — سه نمره

سوال ۱.۱ — بررسی محیط — Docker یک نمره

آنچه می‌خواهم: بررسی کنید که Docker نصب و آماده است.

• فایل اسکریپت ایجاد کنید 01_docker_setup/check_docker.sh :

• نمونه محتوای اسکریپت:

```
#!/bin/bash
```

```
docker --version > docker_version.txt
```

```
docker info | head -n 20 >> docker_version.txt
```

```
docker ps -a >> docker_version.txt
```

```
docker network ls >> docker_version.txt
```

```
docker volume ls >> docker_version.txt
```

- خروجی دستورهای زیر را در 01_docker_setup/docker_status.txt ذخیره کنید:

```
docker system df
```

```
docker network inspect bridge | head -n 30
```

Hints:

اگر Docker نصب نیست، از اسکریپت نصب هفته قبل استفاده کنید

از `docker system df` برای مشاهده استفاده از فضای دیسک استفاده کنید

سوال ۱.۲ Pull — کردن Image های مورد نیاز — دو نمره

- فایل اسکریپت ایجاد کنید `01_docker_setup/pull_images.sh`:

- نمونه محتوای اسکریپت:

```
#!/bin/bash
```

```
docker pull nginx:alpine
```

```
docker pull haproxy:alpine
```

```
docker pull traefik:v3.0
```

```
docker pull python:3.9-alpine
```

```
docker images | grep -E 'nginx|haproxy|traefik|python' > pulled_images.txt
```

- خروجی فایل pulled_images.txt را در پوشه تحویل قرار دهید
- یک فایل 01_docker_setup/images_summary.txt بسازید و حجم هر image را یادداشت کنید

Hints:

از `docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"` برای نمایش مرتب‌تر استفاده کنید

مجموعه سوالات دوم Nginx: در - Docker مبانی — پنج نمره

سوال ۲.۱ — اجرای Nginx ساده در — Container یک نمره

- یک Nginx container ساده اجرا کنید:

```
docker run -d --name nginx-basic -p 8080:80 nginx:alpine
```

- دستورهای زیر را در 02_nginx_basics/basic_nginx.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 2
```

```
curl -I http://localhost:8080 > nginx_response.txt
```

```
docker logs nginx-basic | head -n 10 > nginx_logs.txt
```

```
docker inspect nginx-basic | grep -A 5 "NetworkSettings" > nginx_network.txt
```

```
docker stop nginx-basic
```

```
docker rm nginx-basic
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

از `docker ps` برای بررسی وضعیت container استفاده کنید

از `docker logs` برای مشاهده لاگ‌های Nginx استفاده کنید

سوال ۲.۲ — راه‌اندازی سایت استاتیک با Volume دو نمره

- پوشه `02_nginx_basics/web` بسازید و فایل `index.html` را در آن قرار دهید:

```
<html>
```

```
<body>
```

```
<h1>Demo Site 01</h1>
```

```
<p>Served by Nginx in Docker.</p>
```

```
</body>
```

</html>

- یک container با bind mount اجرا کنید:

```
docker run -d --name nginx-static \
```

```
-p 8080:80 \
```

```
-v $(pwd)/02_nginx_basics/web:/usr/share/nginx/html:ro \
```

```
nginx:alpine
```

- دستورهای زیر را در 02_nginx_basics/static_test.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 2
```

```
curl -i http://localhost:8080/ > static_check.txt
```

```
echo "File modified" >> 02_nginx_basics/web/index.html
```

```
curl -i http://localhost:8080/ >> static_check.txt
```

```
docker stop nginx-static
```

```
docker rm nginx-static
```

- خروجی فایل static_check.txt را در پوشه تحویل قرار دهید

Hints:

با bind mount ، تغییرات در host بلافاصله در container قابل مشاهده است

از ro: برای read-only mount استفاده کنید

سوال ۲.۳ Reverse Proxy — با — Backend Container دو نمره

- یک backend container با Python اجرا کنید:

```
docker run -d --name backend-api \  
--network bridge \  
python:3.9-alpine \  
sh -c "python3 -m http.server 5000"
```

- یک فایل کانفیگ Nginx بسازید 02_nginx_basics/nginx.conf:

```
events {}  
  
http {  
  
    server {  
  
        listen 80;  
  
        location / {  
  
            root /usr/share/nginx/html;  
  
            index index.html;  
  
        }  
  
        location /api {
```

```

        proxy_pass http://backend-api:5000/;

    }

}

}

```

- یک Nginx container با کانفیگ سفارشی اجرا کنید:

```

docker run -d --name nginx-proxy \

-p 8080:80 \

--network bridge \

-v $(pwd)/02_nginx_basics/nginx.conf:/etc/nginx/nginx.conf:ro \

-v $(pwd)/02_nginx_basics/web:/usr/share/nginx/html:ro \

nginx:alpine

```

- دستورهای زیر را در 02_nginx_basics/proxy_test.sh ذخیره و اجرا کنید:

```

#!/bin/bash

sleep 3

curl http://localhost:8080/ > frontend_test.txt

curl http://localhost:8080/api > api_test.txt

docker logs nginx-proxy | tail -n 10 > proxy_logs.txt

docker stop nginx-proxy backend-api

```


`docker rm nginx-proxy backend-api`

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

در Docker network ، container ها می‌توانند با نام container به یکدیگر متصل شوند
از `docker network inspect bridge` برای مشاهده IP های container ها استفاده کنید

مجموعه سوالات سوم HTTPS: و گواهی‌ها در — Docker پنج نمره

سوال ۳.۱ — تولید گواهی — SSL یک نمره

- پوشه `03_nginx_certs/certs` بسازید
- اسکریپت زیر را در `03_nginx_certs/generate_certs.sh` ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
mkdir -p certs
```

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
```

```
-keyout certs/demo.key \
```

```
-out certs/demo.crt \
```

```
-subj "/C=IR/ST=Tehran/L=Tehran/O=DevOpsClass/CN=demo.local"
```

```
ls -lh certs/ > certs_info.txt
```

سوال ۳.۲ — پیکربندی Nginx با HTTPS در Docker سه نمره

- فایل کانفیگ Nginx بسازید nginx_https.conf در مسیر 03_nginx_certs/ :

```
events {}

http {

    server {

        listen 443 ssl;

        server_name demo.local;


        ssl_certificate    /etc/nginx/certs/demo.crt;

        ssl_certificate_key /etc/nginx/certs/demo.key;


        location /status {

            return 200 "HTTPS OK";

            add_header Content-Type text/plain;

        }

    }

}
```

- یک container با volume mount برای گواهی‌ها اجرا کنید:

```
docker run -d --name nginx-https \  
  
-p 8443:443 \  
  
-v $(pwd)/03_nginx_certs/nginx_https.conf:/etc/nginx/nginx.conf:ro \  
  
-v $(pwd)/03_nginx_certs/certs:/etc/nginx/certs:ro \  
  
nginx:alpine
```

- دستورهای زیر را در 03_nginx_certs/https_test.sh ذخیره و اجرا کنید:

```
#!/bin/bash  
  
sleep 2  
  
curl -k https://localhost:8443/status > https_response.txt  
  
openssl s_client -connect localhost:8443 -servername demo.local < /dev/null  
2>&1 | \  
  
openssl x509 -noout -dates >> https_response.txt  
  
docker logs nginx-https | tail -n 10 > https_logs.txt  
  
docker stop nginx-https  
  
docker rm nginx-https
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

از k در curl برای ignore کردن certificate validation استفاده کنید

گواهی‌ها باید در container mount شوند

سوال ۳.۳ — مقایسه HTTP و HTTPS — یک نمره

- یک فایل 03_nginx_certs/http_vs_https.txt بسازید و به این سوالات پاسخ دهید:

۱. تفاوت اصلی HTTP و HTTPS چیست؟

۲. چرا در Docker باید پورت‌های مختلف برای HTTP و HTTPS استفاده کنیم؟

۳. چگونه می‌توانید هم HTTP و هم HTTPS را در یک container فعال کنید؟

- خروجی دستور زیر را نیز اضافه کنید:

```
docker port nginx-https
```

```
ss -tuln | grep -E ':8443'
```

Hints:

- از docker port برای مشاهده port mapping استفاده کنید

مجموعه سوالات چهارم: مانیتورینگ و مدیریت Nginx در Docker — چهار نمره

سوال ۴.۱ — فعال کردن Status Module — دو نمره

- فایل کانفیگ Nginx بسازید 04_nginx_admin/nginx_status.conf :

```

events {}

http {

    server {

        listen 80;

        location /basic_status {

            stub_status;

            allow all;

        }

        location / {

            return 200 "Nginx Status Page";

            add_header Content-Type text/plain;

        }

    }

}

```

- یک container اجرا کنید:

```

docker run -d --name nginx-status \

-p 8090:80 \

-v $(pwd)/04_nginx_admin/nginx_status.conf:/etc/nginx/nginx.conf:ro \

```

nginx:alpine

- دستورهای زیر را در 04_nginx_admin/status_test.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 2
```

#چند درخواست برای تولید ترافیک

```
for i in {1..5}; do
```

```
    curl http://localhost:8090/ > /dev/null
```

```
done
```

```
curl http://localhost:8090/basic_status > status_output.txt
```

```
docker logs nginx-status | tail -n 15 > status_logs.txt
```

```
docker stop nginx-status
```

```
docker rm nginx-status
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

stub_status اطلاعات آماری Nginx را نمایش می‌دهد

از allow all فقط برای تست استفاده کنید، در production محدود کنید

سوال ۴.۲ — مانیتورینگ لاگ‌های Container — دو نمره

- یک container با لاگ‌های بیشتر اجرا کنید:

```
docker run -d --name nginx-logs \
```

```
-p 8091:80 \
```

```
nginx:alpine
```

- دستورهای زیر را در 04_nginx_admin/log_monitoring.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 2
```

#تولید ترافیک

```
curl http://localhost:8091/ > /dev/null
```

```
curl http://localhost:8091/nonexistent 2>&1 > /dev/null
```

#استخراج لاگ‌ها

```
docker logs nginx-logs --tail 20 > latest_logs.txt
```

```
docker logs nginx-logs 2>&1 | grep -i error > error_logs.txt || echo "No  
errors found" > error_logs.txt
```

#بررسی لاگ‌های real-time

```
docker logs nginx-logs --since 1m > recent_logs.txt
```

`docker stop nginx-logs`

`docker rm nginx-logs`

- یک فایل `04_nginx_admin/log_analysis.txt` بسازید و به این سوالات پاسخ دهید:

۱. تفاوت `docker logs` و لاگ‌های فایل سیستم چیست؟

۲. چگونه می‌توانید لاگ‌های `container` را به فایل سیستم `redirect` کنید؟

۳. مزایای استفاده از `docker logs` چیست؟

Hints:

از `docker logs -f` برای دنبال کردن `real-time` استفاده کنید (اما در اسکرپت نباشد)

از `since` و `until` برای فیلتر زمانی استفاده کنید

مجموعه سوالات پنجم mTLS در Nginx با Docker — پنج نمره

سوال ۵.۱ — تولید گواهی‌های mTLS — یک نمره

- اسکرپت زیر را در `05_nginx_mtls/generate_certs.sh` ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
set -e
```



```
mkdir -p ca client server
```

#گواهی CA

```
openssl req -x509 -new -nodes -keyout ca/rootCA.key -sha256 -days 365 \  
-out ca/rootCA.crt -subj "/C=IR/O=DevOpsClass/CN=demo-ca"
```

#گواهی سرور

```
openssl req -new -nodes -out server/server.csr -keyout server/server.key \  
-subj "/C=IR/O=DevOpsClass/CN=mtls.local"
```

```
openssl x509 -req -in server/server.csr -CA ca/rootCA.crt -CAkey  
ca/rootCA.key \  

```

```
-out server/server.crt -days 365 -sha256 -CAcreateserial
```

#گواهی کلاینت

```
openssl req -new -nodes -out client/client.csr -keyout client/client.key \  
-subj "/C=IR/O=DevOpsClass/CN=client1"
```

```
openssl x509 -req -in client/client.csr -CA ca/rootCA.crt -CAkey  
ca/rootCA.key \  

```

```
-out client/client.crt -days 365 -sha256 -CAcreateserial
```

```
echo "Certificates generated successfully"
```

```
ls -R . > certs_structure.txt
```

سوال ۵.۲ — پیکربندی mTLS در Nginx Container سه نمره

- فایل کانفیگ Nginx بسازید 05_nginx_mtls/nginx_mtls.conf :

```
events {}
```

```
http {
```

```
    server {
```

```
        listen 9443 ssl;
```

```
        server_name mtls.local;
```

```
        ssl_certificate    /etc/nginx/mtls/server/server.crt;
```

```
        ssl_certificate_key /etc/nginx/mtls/server/server.key;
```

```
        ssl_client_certificate /etc/nginx/mtls/ca/rootCA.crt;
```

```
        ssl_verify_client on;
```

```
        location / {
```

```

        return 200 "mTLS OK\n";

        add_header Content-Type text/plain;

    }

}

}

```

یک container با mTLS اجرا کنید:

```

docker run -d --name nginx-mtls \

-p 9443:9443 \

-v $(pwd)/05_nginx_mtls/nginx_mtls.conf:/etc/nginx/nginx.conf:ro \

-v $(pwd)/05_nginx_mtls:/etc/nginx/mtls:ro \

nginx:alpine

```

- دستورهای زیر را در 05_nginx_mtls/mtls_test.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 2
```

#تست بدون گواهی کلاینت) باید fail شود)

```
echo "=== Test without client certificate ===" > mtls_test.txt
```

```
curl -k https://localhost:9443/ -v 2>&1 | grep -E 'HTTP|SSL|certificate' >>
mtls_test.txt
```

#تست با گواهی کلاینت (باید موفق شود)

```
echo -e "\n=== Test with client certificate ===" >> mtls_test.txt
```

```
curl -k https://localhost:9443/ \
```

```
--cert client/client.crt \
```

```
--key client/client.key \
```

```
-v 2>&1 | grep -E 'HTTP|SSL|certificate' >> mtls_test.txt
```

```
docker logs nginx-mtls | tail -n 10 > mtls_logs.txt
```

```
docker stop nginx-mtls
```

```
docker rm nginx-mtls
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

انتظار دارید درخواست اول با خطای SSL رد شود

درخواست دوم با گواهی کلاینت باید موفق شود

سوال ۵.۳ — توضیح mTLS و — Certificate Chain یک نمره

- یک فایل 05_nginx_mtls/mtls_explanation.txt بسازید و به این سوالات پاسخ دهید:

۱. تفاوت TLS و mTLS چیست؟

۲. چرا کلاینت باید rootCA.crt را بشناسد؟

۳. در Docker، چگونه می‌توانید گواهی‌ها را به صورت secure مدیریت کنید؟

- دستور زیر را هم اجرا و خروجی را در همان فایل قرار دهید:

```
openssl verify -CAfile ca/rootCA.crt client/client.crt
```

Hints:

از Markdown برای خوانا تر بودن استفاده کنید

درباره Docker secrets و volume management توضیح دهید

مجموعه سوالات ششم HAProxy در — Docker چهار نمره

سوال ۶.۱ — راه‌اندازی Backend Containers — یک نمره

- دو backend container با محتوای متفاوت اجرا کنید:

```
# Backend 1
```

```
mkdir -p 06_haproxy/backend1
```

```
echo "<h1>Backend Server 1</h1>" > 06_haproxy/backend1/index.html
```

```
docker run -d --name backend1 \
```

```
--network bridge \  
  
-v $(pwd)/06_haproxy/backend1:/usr/share/nginx/html:ro \  
  
-p 8080:80 \  
  
nginx:alpine
```

Backend 2

```
mkdir -p 06_haproxy/backend2  
  
echo "<h1>Backend Server 2</h1>" > 06_haproxy/backend2/index.html  
  
docker run -d --name backend2 \  
  
--network bridge \  
  
-v $(pwd)/06_haproxy/backend2:/usr/share/nginx/html:ro \  
  
-p 8081:80 \  
  
nginx:alpine
```

- دستورهای زیر را در 06_haproxy/setup_backends.sh ذخیره و اجرا کنید:

```
#!/bin/bash  
  
sleep 2  
  
curl http://localhost:8080/ > backend1_response.txt  
  
curl http://localhost:8081/ > backend2_response.txt
```

```
docker ps | grep backend > backends_status.txt
```

سوال ۶.۲ — پیکربندی HAProxy در Docker — سه نمره

- فایل کانفیگ HAProxy بسازید 06_haproxy/haproxy.cfg :

```
global
```

```
    daemon
```

```
    maxconn 256
```

```
    log stdout format raw local0
```

```
defaults
```

```
    mode http
```

```
    timeout connect 5000ms
```

```
    timeout client 50000ms
```

```
    timeout server 50000ms
```

```
    log global
```

```
frontend http_front
```

```
    bind *:9000
```

```
    default_backend http_back
```

```
backend http_back
```

```
balance roundrobin
```

```
server server1 backend1:80 check
```

```
server server2 backend2:80 check
```

- یک container HAProxy اجرا کنید:

```
docker run -d --name haproxy-lb \
```

```
-p 9000:9000 \
```

```
--network bridge \
```

```
-v $(pwd)/06_haproxy/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro \
```

```
haproxy:alpine
```

- دستورهای زیر را در 06_haproxy/haproxy_test.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 3
```

#تست - *load balancing* چند درخواست بزنید

```
echo "=== Load Balancing Test ===" > haproxy_test.log
```

```
for i in {1..6}; do
```



```
echo "Request $i:" >> haproxy_test.log
```

```
curl http://localhost:9000/ >> haproxy_test.log
```

```
echo "" >> haproxy_test.log
```

done

```
docker logs haproxy-lb | tail -n 20 > haproxy_logs.txt
```

```
docker stats --no-stream haproxy-lb backend1 backend2 > haproxy_stats.txt
```

```
docker stop haproxy-lb backend1 backend2
```

```
docker rm haproxy-lb backend1 backend2
```

- یک فایل 06_haproxy/load_balancing_explanation.txt بسازید و به این سوالات پاسخ دهید:

۱. الگوریتم load balancing استفاده شده چیست؟

۲. چگونه HAProxy health check انجام می‌دهد؟

۳. تفاوت HAProxy و Nginx به عنوان load balancer چیست؟

Hints:

در Docker network ، از نام container برای اتصال استفاده کنید

از balance roundrobin برای توزیع مساوی استفاده می‌شود

مجموعه سوالات هفتم Traefik با Docker — چهار نمره

سوال ۷.۱ — راه‌اندازی Traefik با Docker Provider — دو نمره

- فایل docker-compose.yml بسازید: 07_traefik/docker-compose.yml

```
version: '3.8'
```

```
services:
```

```
traefik:
```

```
image: traefik:v3.0
```

```
command:
```

- "--api.insecure=true"
- "--providers.docker=true"
- "--providers.docker.exposedbydefault=false"
- "--entrypoints.web.address=:80"

```
ports:
```

- "8080:80"

- "8081:8080" # *Dashboard*

volumes:

- /var/run/docker.sock:/var/run/docker.sock:ro

networks:

- traefik-net

whoami:

image: traefik/whoami

labels:

- "traefik.enable=true"

- "traefik.http.routers.whoami.rule=Host(`localhost`)"

- "traefik.http.routers.whoami.entrypoints=web"

networks:

- traefik-net

networks:

traefik-net:

driver: bridge

- دستورهای زیر را در 07_traefik/run_traefik.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
cd 07_traefik
```

```
docker-compose up -d
```

```
sleep 5
```

```
docker-compose ps > traefik_status.txt
```

```
curl http://localhost:8080/ > whoami_response.txt
```

```
curl http://localhost:8081/api/rawdata > traefik_api.txt 2>&1 || echo "API  
check" >> traefik_api.txt
```

```
docker-compose logs traefik | tail -n 30 > traefik_logs.txt
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

Traefik از Docker socket برای discovery استفاده می‌کند

Labels در docker-compose برای پیکربندی routing استفاده می‌شوند

سوال ۷.۲ — پیکربندی پیشرفته — Traefik دو نمره

- فایل docker-compose.yml را به‌روزرسانی کنید تا شامل چند سرویس باشد
07_traefik/docker-compose-advanced.yml :

```
version: '3.8'
```

services:

traefik:

image: traefik:v3.0

command:

- "--api.insecure=true"
- "--providers.docker=true"
- "--providers.docker.exposedbydefault=false"
- "--entrypoints.web.address=:80"
- "--entrypoints.websecure.address=:443"

ports:

- "8080:80"
- "8443:443"
- "8081:8080"

volumes:

- /var/run/docker.sock:/var/run/docker.sock:ro

networks:

- traefik-net

web1:

image: nginx:alpine

labels:

- "traefik.enable=true"
- "traefik.http.routers.web1.rule=Host(`web1.localhost`)"
- "traefik.http.routers.web1.entrypoints=web"

networks:

- traefik-net

web2:

image: nginx:alpine

labels:

- "traefik.enable=true"
- "traefik.http.routers.web2.rule=Host(`web2.localhost`)"
- "traefik.http.routers.web2.entrypoints=web"

networks:

- traefik-net

networks:

traefik-net:

driver: bridge

- دستورهای زیر را در 07_traefik/advanced_test.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
cd 07_traefik
```

```
docker-compose -f docker-compose-advanced.yml up -d
```

```
sleep 5
```

```
echo "=== Testing web1 ===" > advanced_test.txt
```

```
curl -H "Host: web1.localhost" http://localhost:8080/ >> advanced_test.txt
```

```
echo -e "\n=== Testing web2 ===" >> advanced_test.txt
```

```
curl -H "Host: web2.localhost" http://localhost:8080/ >> advanced_test.txt
```

```
echo -e "\n=== Traefik Dashboard ===" >> advanced_test.txt
```

```
curl http://localhost:8081/api/http/routers >> advanced_test.txt
```

docker-compose -f docker-compose-advanced.yml down

- یک فایل 07_traefik/traefik_explanation.txt بسازید و به این سوالات پاسخ دهید:

۱. مزایای استفاده از Traefik با Docker چیست؟

۲. چگونه Traefik به صورت خودکار routing را تشخیص می‌دهد؟

۳. تفاوت Traefik و Nginx/HAProxy در چیست؟

Hints:

- Traefik از Docker labels برای automatic service discovery استفاده می‌کند

- از "Host: ..." برای تست virtual hosting استفاده کنید

تحويل نهایی و فایل خروجی — یک نمره

- یک فایل README.md در ریشه پوشه تمرین قرار دهید و شامل موارد زیر باشد:

۱. توضیح کوتاه پروژه

۲. نحوه راه‌اندازی (دستورات docker و docker-compose)

۳. ساختار فایل‌ها و نقش هر بخش

۴. نحوه دسترسی به سرویس‌ها (پورت‌ها و URL ها)

- خروجی دستور tree یا لیست find را در فایل final_structure.txt ذخیره کنید
 - یک فایل docker_commands_history.txt شامل تمام دستورات Docker که اجرا کرده‌اید (می‌توانید از grep docker | history استفاده کنید)
-

معیار نمره‌دهی خلاصه

- اجرای صحیح container ها و docker-compose: سی درصد
 - کار با volumes ، networks و port mapping: بیست درصد
 - پیکربندی صحیح: reverse proxy (Nginx, HAProxy, Traefik): بیست درصد
 - رعایت دقیق مسیرها و نام فایل‌ها: ده درصد
 - توضیحات کوتاه و مرتب در فایل‌های متنی: ده درصد
 - سازمان‌دهی پوشه و تحویل تمیز: ده درصد
- مهلت تحویل: جمعه هفته آینده